РЕКЛАМА
При поддержке Хабр Курсы РЕКЛАМА
Хабр Курсы
Взлетайте в профессии

Creatman
3 часа назад

Bug fingerprinting для UI: почему stack trace не работает и что вместо

Сложный 8 мин 4.3K

Тестирование веб-сервисов*, Программирование*, Алгоритмы*, Open source*, Искусственный интеллект

Аналитика

TL;DR: Sentry дедуплицирует backend-ошибки по хешу `(error class + top stack frame + module)`. Для UI-багов этот рецепт ломается — у `expect(button).toBeVisible()` нет stack frame в продуктовом смысле, есть **локатор + assertion + URL**. В webtest-orch я собрал composite SHA-256 fingerprint из `(normalized_selector | assertion type | error class | URL template | message[:80])` с тремя rules нормализации (`:nth-child`, UUID, `/users/123 → /users/:id`). Это даёт стабильный 8-hex `BUG-id` который выживает прогоны и даёт diff `new / regression / persisting / fixed` без БД и embedding'ов.

Сразу прошу читателя простить меня но дальше будет много букав и кода.

Когда я писал webtest-orch — Claude Code skill для e2e-тестирования — упёрся в задачу которая на первый взгляд тривиальная. Тест провалился. На следующем прогоне — снова. Это **тот же баг** или **новый**? Если тот же — на каком прогоне он впервые появился, и был ли он fixed между ними? Если ответа нет, run-diff невозможен, и любая регрессия теряется в шуме «9 fail'ов сегодня».

Backend-задачу решил Sentry — composite hash из error class и top stack frame. UI-багам этот рецепт не подходит, и в этой статье я разберу почему, и какой fingerprint в итоге работает.

Почему рецепт Sentry не подходит для UI

Sentry's grouping algorithm дедуплицирует backend-ошибки через комбинацию:

- Тип исключения (`TypeError`, `NullPointerException`)
- Top frame stack trace (`module.py:42 → user_handler`)
- Транзитивно — содержание top функции

Это работает потому что у backend-исключения есть детерминированный stack — одна функция, одна строка, одно исключение.

UI-баги выглядят иначе. Падает test:

При поддержке Хабр Курсы РЕКЛАМА

```
Error: expect(locator).toBeVisible() failed
Locator: getByRole('button', { name: 'Place Order' })
Expected: visible
Actual: not found
URL: https://shop.example.com/checkout
```

Объяснить с SourceCraft

Какой здесь “top stack frame”? Технически — Playwright internals (`expect.js:128`), но для дедупликации это **бесполезный носитель**. Регрессии будут все из одного и того же `expect.js:128` , потому что Playwright использует одну функцию для всех `expect(locator).toBeVisible()` в мире.

То что **на самом деле уникально** для каждого UI-бага:

1. **Что мы искали** — locator (`getByRole('button', { name: 'Place Order' })`)
2. **Что мы проверяли** — assertion (`toBeVisible` , `toHaveText` , `toHaveURL`)
3. **Тип ошибки** — class (`TimeoutError` , `AssertionError`)
4. **Где** — URL pattern (без волатильных частей)
5. **Содержание** — short snippet of error message

Эти пять полей в комбинации идентифицируют bug semantically — не его technical signature, а его **product meaning**.

Naive подход и почему он ломается

Первая итерация — конкатенация всех пяти полей:

```
fingerprint = sha256(f"{selector}|{assertion}|{error_class}|{url}|{message}")[:8]
```

Объяснить с

Это работает на 80% случаев. Рушится на остальных 20%, и эти 20% — самые информативные.

Случай 1 — `:nth-child(N)`

Тест воспроизводит баг на третьем элементе списка. Selector:

```
getByRole('button').nth(2)  // i.e. nth-child(3) in DOM
```

Объяснить с

Завтра кто-то добавил элемент в начало списка — теперь баг на четвёртом. Selector:

```
getByRole('button').nth(3)  // i.e. nth-child(4) in DOM
```

Объяснить с

Это **тот же баг** (кнопка с тем же accessible name не работает в том же UI-контексте), но naive fingerprint видит два разных. Результат — каждое изменение в списке создаёт «новый» bug. Run-diff

При поддержке Хабр Курсы РЕКЛАМА

Случай 2 — UUIDs в URL

```
URL: /run/abc123de-1234-1234-1234-123456789abc/details
```

Объяснить с

Каждый прогон — новый UUID. Каждый прогон — новый fingerprint. Каждый баг “новый” forever.

Случай 3 — числовые ID в URL

```
URL: /users/12345/profile
```

Объяснить с

Тестовый юзер генерируется в setup project → разный ID каждый прогон. Тот же fingerprint problem.

Случай 4 — query strings

```
URL: /search?q=foo&utm_source=test&_=1715693845
```

Объяснить с

Timestamp в query, UTM-параметры из ad campaign на тестовой среде. Fingerprint меняется каждый раз когда запускаешь.

Решение — нормализация перед хешированием

```python
import re
import hashlib

SELECTOR_NORMS = [
    (re.compile(r":nth-child\(\d+\)"),   ":nth-child"),
    (re.compile(r":nth-of-type\(\d+\)"), ":nth-of-type"),
]

URL_NORMS = [
    # UUIDs (любая позиция)
    (re.compile(r"[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}", re.I),
     ":uuid"),
    # Числовые ID между слешами
    (re.compile(r"/\d+(?=/|$|\?)"), "/:id"),
    # Query strings — целиком
    (re.compile(r"\?.*$"), ""),
]


def normalize_selector(s: str) -> str:
    for pat, repl in SELECTOR_NORMS:
        s = pat.sub(repl, s)
    return s
```

При поддержке Хабр Курсы РЕКЛАМА

```python
def normalize_url(url: str) -> str:
    if not url:
        return ""
    # Stripping protocol+host если есть
    m = re.match(r"^https?://[^/]+(.*)", url)
    path = m.group(1) if m else url
    for pat, repl in URL_NORMS:
        path = pat.sub(repl, path)
    return path


def compute_fingerprint(bug: dict) -> str:
    err = bug.get("error") or {}
    msg = (err.get("message") or "")[:120]
    selector = normalize_selector(bug.get("selector", ""))
    assertion = bug.get("assertion_type", "Generic")
    error_class = bug.get("error_class", "AssertionError")
    url_path = normalize_url(err.get("url") or "")

    composite = f"{selector}|{assertion}|{error_class}|{url_path}|{msg[:80]}"
    return hashlib.sha256(composite.encode("utf-8")).hexdigest()[:8]
```

Объяснить с

После этих трёх правил:

До нормализации	После
`getByRole('button').nth(2)`	`getByRole('button').nth(2)` (nth не трогаем — это explicit пользовательский intent)
`getByRole('item') :nth-child(3)`	`getByRole('item') :nth-child`
`/run/abc123de-...-12345.../info`	`/run/:uuid/info`
`/users/12345/profile`	`/users/:id/profile`
`/search?q=x&utm=y`	`/search`

`nth(N)` (Playwright explicit API) специально не нормализуем — это intent программиста: «именно третья кнопка». А `:nth-child(N)` (CSS селектор) нормализуем, потому что туда обычно попадает positional index из автогенерированного селектора, который шумит между прогонами.

Длина fingerprint — 8 hex chars (32 бита). При коллизионной вероятности ~2^-32 на пару, для типичного suite в 50-100 багов это даёт <0.001% false collision rate. SHA-256 «full» использовать смысла нет — короткий fingerprint удобнее для UI отчётов (`BUG-a3f9c2b1`).

Извлечение полей из разнотипных runtime-ошибок

Selector редко лежит в structured field. Playwright кидает его в error.message + snippet. Вот regex-extractor для популярных Playwright-локаторов:

При поддержке Хабр Курсы РЕКЛАМА

```python
def extract_selector(error_snippet: str, error_msg: str) -> str:
    haystack = (error_snippet or "") + " " + (error_msg or "")
    for prefix in ("getByRole", "getByLabel", "getByText", "getByTestId",
                   "getByPlaceholder", "getByAltText", "getByTitle", "locator"):
        m = re.search(rf"{prefix}\([^)]+\)", haystack)
        if m:
            return m.group(0)
    return ""


def extract_assertion_type(msg: str) -> str:
    for kw in ("toBeVisible", "toHaveText", "toHaveURL", "toHaveTitle",
               "toHaveAttribute", "toContainText", "toEqual", "toBe"):
        if kw in msg:
            return kw
    if re.search(r"timeout|Timeout", msg):
        return "Timeout"
    return "Generic"


def extract_error_class(msg: str) -> str:
    m = re.match(r"^([A-Z][A-Za-z]+(Error|Exception)):", msg)
    if m:
        return m.group(1)
    if re.search(r"timeout|Timeout", msg):
        return "TimeoutError"
    return "AssertionError"
```

Объяснить с

Парсер выбирает первый match из приоритетного списка. `getByRole` идёт первым потому что он чаще всего используется и лучше всего describes intent. `locator(...)` — fallback для кастомных селекторов.

Один edge case: если в error message несколько локаторов (например, `expect(getByRole('button')).toContain(getByText('saved'))`) — берётся первый. Это конвенция, не optimal, но **стабильна** между прогонами того же теста.

Run-diff: что делать с fingerprint'ом

Сам по себе fingerprint полезен только для дедупликации внутри одного прогона. Реальный value — diff между прогонами. Алгоритм:

```python
def diff_runs(current: list[dict], previous: list[dict]) -> dict:
    prev_map = {b.get("fingerprintHash"): b for b in previous if b.get("fingerprintHash
    summary = {"new": 0, "regression": 0, "persisting": 0, "fixed": 0}
    out = []
    cur_hashes = set()

    for bug in current:
        fp = bug["fingerprintHash"]
        cur_hashes.add(fp)
        prev = prev_map.get(fp)
```

При поддержке Хабр Курсы РЕКЛАМА

```
            state = "new"
        elif (prev.get("diff") or {}).get("state") == "fixed":
            state = "regression"  # был fixed, вернулся
        else:
            state = "persisting"
            bug["occurrenceCount"] = (prev.get("occurrenceCount") or 1) + 1

        bug["diff"] = {
            "state": state,
            "previousRunId": prev.get("lastSeenRunId") if prev else None,
        }
        summary[state] += 1
        out.append(bug)

    # Newly fixed: present in previous, absent in current
    for fp, prev in prev_map.items():
        if fp in cur_hashes:
            continue
        if (prev.get("diff") or {}).get("state") == "fixed":
            continue  # already fixed earlier — не считаем "newly"
        prev_copy = dict(prev)
        prev_copy["diff"] = {"state": "fixed", "previousRunId": prev.get("lastSeenRunI
        out.append(prev_copy)
        summary["fixed"] += 1

    return {"bugs": out, "summary": summary}
```

Объяснить с

Состояния:

- **`new`** — fingerprint не было в предыдущем прогоне
- **`persisting`** — был, всё ещё есть; счётчик occurrence инкрементится
- **`fixed`** — был в предыдущем, в текущем отсутствует, и НЕ был помечен fixed раньше
- **`regression`** — был, был помечен fixed, потом вернулся

`regression` — самое важное состояние. Это сигнал что починка сломалась и нужно investigate. В отчёте webtest-orch — 🚨 `Regression` в diff-секции `report.md`.

Edge cases которые хочется упомянуть

Viewport-dependent bugs. `touch-target: BUTTON 86×20 на 390×844` — fingerprint включает viewport size. Если запускать на mobile **и** desktop, тот же баг имеет два разных fingerprint'а. Это правильное поведение — баг существует только на mobile, не на desktop. Если хочется коллапсировать в один — добавить ещё одну норм.правило: `r"\d+x\d+" → "WxH"`. Это уже зависит от того как ты хочешь видеть отчёт.

Selector с переменным текстом. Динамические welcome-сообщения вроде `getByText('Welcome, John')` создают fingerprint per username. Решение — переписывать spec на `getByRole('heading', { name: /Welcome/ })` ещё на стадии генерации, не в fingerprint'е. Fingerprint надёжен только когда spec написан правильно.

При поддержке Хабр Курсы РЕКЛАМА

`issues` содержит 8 axe-core violations, fingerprint тестовой ошибки — один. Поэтому в webtest-orch `run_ suite.py` сначала **расщепляет issues collector на отдельные bug records**, потом fingerprint каждый отдельно. Это важная архитектурная деталь — без неё все a11y-violations одной страницы дают один `BUG-id` и теряются индивидуальные регрессии.

SHA-256 vs xxhash vs CRC32. Я выбрал SHA-256 по двум причинам: (1) криптостойкость не нужна, но коллизии важны — на 32 бита SHA-256 распределяется лучше CRC32 на типичных коротких строках; (2) std lib в Python без зависимостей. xxhash был бы быстрее, но для 100-1000 багов в прогоне разница — миллисекунды.

Severity inference как side-effect

Из тех же extracted полей можно вывести severity без LLM. axe-core impact'ы маппятся напрямую:

```
def severity_from_signals(bug: dict) -> str:
    issue_line = (bug.get("issueLine") or "").lower()

    # axe-core impact → severity
    if "a11y[critical]" in issue_line or "a11y[serious]" in issue_line:
        return "S1"
    if "a11y[moderate]" in issue_line:
        return "S2"
    if "a11y[minor]" in issue_line:
        return "S3"

    # Структурные теги от spec template
    if issue_line.startswith("heading-jump:"):     return "S2"
    if issue_line.startswith("touch-target:"):     return "S2"
    if issue_line.startswith("overflow:"):         return "S1"

    # Heuristic match на keywords из title + error
    text = (bug.get("title", "") + " " + bug.get("error", {}).get("message", "")).lower
    if any(k in text for k in ["auth", "login", "checkout", "payment", "5xx"]):
        return "S0"
    if any(k in text for k in ["uncaught", "hydration"]):
        return "S0"

    return "S2"  # default
```

Объяснить с

Эта эвристика покрывает 80% случаев. Для остальных 20% (P0 product regression со generic-сообщением) добавлен override mechanism через комментарий перед `test()`:

```
// @severity: S0
test('checkout completely broken', async ({ page }) => { /* ... */ });
```

Объяснить с

`fingerprint_ bugs.py` парсит spec файл, ищет `// @severity:` комментарии перед `test()`, матчит по test title и применяет override. Это решает false-negative case когда heuristic недооценивает критичность.

При поддержке Хабр Курсы РЕКЛАМА

Что получается на выходе

`bugs.json` после прогона:

```
{
  "runId": "run-2026-04-30-1430",
  "bugs": [
    {
      "id": "BUG-a3f9c2b1",
      "fingerprintHash": "a3f9c2b1",
      "title": "Checkout 'Place Order' button non-functional on mobile",
      "severity": "S0",
      "priority": "P0",
      "diff": { "state": "regression", "previousRunId": "run-2026-04-26-0902" },
      "occurrenceCount": 3,
      "trackerMappings": {
        "linear": { "priority": 1 },
        "github": { "labels": ["bug", "severity/s0", "priority/p0"] },
        "jira":   { "issueType": "Bug", "priorityName": "Highest" }
      }
    }
  ]
}
```

Объяснить с

Из этого структуры родится auto-filing в трекеры (Linear/GitHub/Jira), markdown-отчёт с severity breakdown, run-diff summary в `diff.json`. Всё детерминированно, без LLM в pipeline — экономно и воспроизводимо.

Полная имплементация — в webtest-orch, скрипт `scripts/fingerprint_bugs.py`, ~250 строк, MIT.

Полезное

- webtest-orch — Claude Code skill для e2e-тестирования, использует этот fingerprint
- Sentry grouping docs — оригинальный backend-подход на котором этот UI-вариант построен по аналогии
- Playwright locator priority — почему `getByRole` идёт первым в нашем extractor

— Nick (Creatman). Делаю инструменты для разработчиков на Claude Code: webtest-orch, cc-janitor, claude-code-antiregression-setup, claude-statusline, ai-context-hierarchy, notebooklm-claude-workflows. Все MIT, все на github.com/CreatmanCEO.

Теги: playwright, qa automation, qa testing, claude, agents, agent skills, aria, web-разработка, fingerprint, ui

Хабы: Тестирование веб-сервисов, Программирование, Алгоритмы, Open source, Искусственный интеллект

При поддержке Хабр Курсы РЕКЛАМА

0 2 0

Редакторский дайджест

Присылаем лучшие статьи раз в месяц

Электропочта

Подписаться

Оставляя почту, я принимаю Политику конфиденциальности и даю согласие на получение рассылок

16K+ Охват за 30 дней

12 Карма

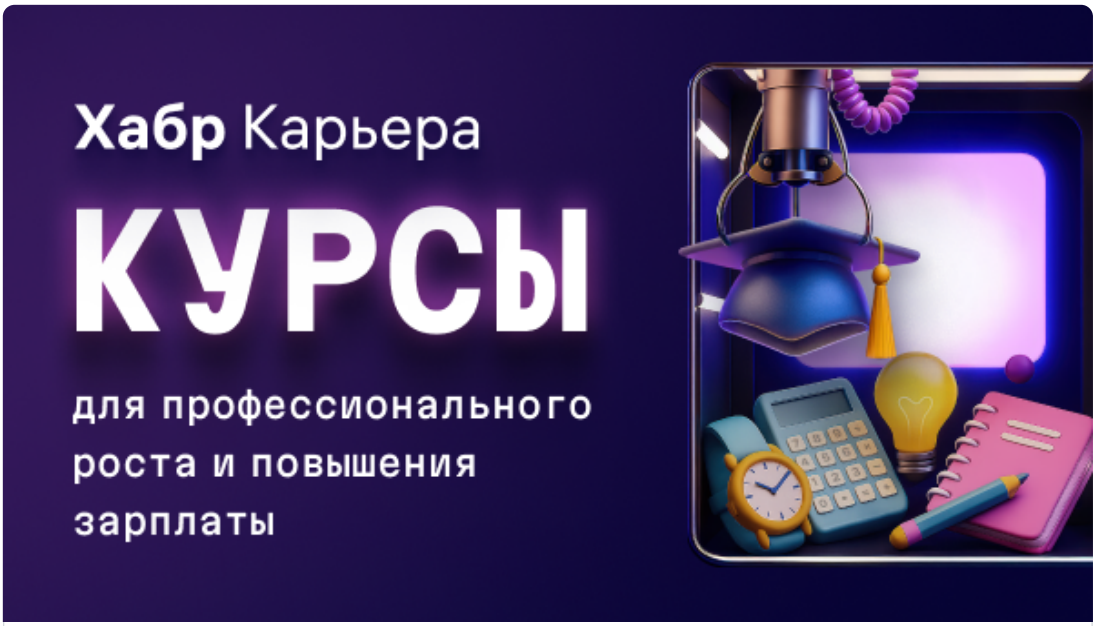
Nick @Creatman

Соло разработчик, CEO в CREATMAN

Подписаться

Сайт GitHub

Поток AI и ML доступен 24/7 благодаря поддержке друзей Хабра



Хабр Курсы для всех РЕКЛАМА

Практикум, Хекслет, SkyPro, авторские курсы — собрали всех и попросили скидки. Осталось выбрать!

Перейти

Перейти в поток AI и ML

Комментарии

Здесь пока нет ни одного комментария, вы можете стать первым!

Зарегистрируйтесь на Хабре, чтобы оставить комментарий

При поддержке Хабр Курсы РЕКЛАМА

Публикации

ЛУЧШИЕ ЗА СУТКИ ПОХОЖИЕ

samako
8 часов назад

Давайте уже сделаем отдельный хаб: «Я собрал приложение за вечер — зачем теперь разработчики?»

Простой 2 мин 9.7K

Мнение

+97 9 45

inkedsymon
11 часов назад

Муравьи против трансформеров: старый алгоритм 1992 года, который вернулся

Средний 9 мин 8.7K

Обзор

+55 35 1

da1ight
5 часов назад

Голосуй, или проиграешь! разбор фишинга, который пытается угнать Telegram

Средний 13 мин 7.8K

Кейс

+30 13 16

strannik96
10 часов назад

Чебурнет близко

Простой 5 мин 9K

Обзор

+29 22 56

techno_mot
7 часов назад

Люди скупают Mac Mini M4, а SpaceX покупает Cursor за $60 млрд: ML-дайджест

8 мин 6.7K

Дайджест

+28 5 2

raipc
10 часов назад

При поддержке Хабр Курсы РЕКЛАМА

в observability-платформе

12 мин 7.1K

+25 6 0

programmerguru
12 часов назад

Как запускать LLM-агентов без облачных API: VPS, локальные модели и требования к железу

Средний 6 мин 10K

Туториал

+23 45 4

PatientZero
13 часов назад

Баги, которые не ловит Rust

Средний 12 мин 9K

Перевод

+23 22 0

morett1m
12 часов назад

Что делает match после того, как вы нажали Compile

Средний 11 мин 7.6K

Обзор

+22 17 1

AnSa8
5 часов назад

OneOCR — скрытая OCR внутри Windows 11

Простой 7 мин 6.2K

Туториал Из песочницы

+21 8 2

Big Data в фарме: как мы управляем сотнями сотрудников по всей стране

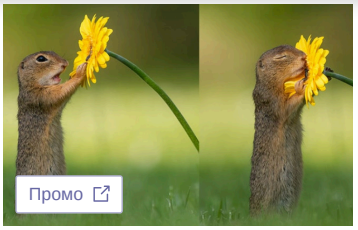
Турбо

Показать еще

МИНУТОЧКУ ВНИМАНИЯ

При поддержке **Хабр** Курсы РЕКЛАМА

Записи Хабр Семинара про смысл работы



Запахло весной и скидками в Промокодусе

Удалёнка в фарме: большие данные и никакой магии

КУРСЫ

Data Scientist с нуля
По мере набора группы

Python-разработчик
По мере набора группы

Профессия: Python-разработчик + ИИ
По мере набора группы

Алгоритмы и структуры данных
По мере набора группы

ИИ для бизнеса
По мере набора группы

Больше курсов на Хабр Карьере

При поддержке Хабр Курсы РЕКЛАМА

При поддержке Хабр Курсы РЕКЛАМА

ЧИТАЮТ СЕЙЧАС

Представлен открытый проект RKN Block Checker

30K 33

Крупные разработчики игр и IT-специалисты бегут из России

65K 261

У чат-бота Grok в X украли $175 тыс. с помощью азбуки Морзе

14K 22

«Аэрофлот» изменил правила провоза пауэрбанков на своих рейсах

5K 12

Радар для слежения за БПЛА. Часть 1

6.3K 4

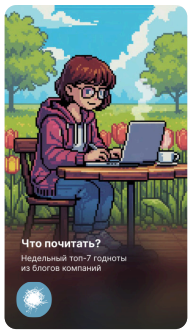
Big Data в фарме: как мы управляем сотнями сотрудников по всей стране

Турбо

При поддержке Хабр Курсы РЕКЛАМА

При поддержке Хабр Курсы РЕКЛАМА

ИСТОРИИ

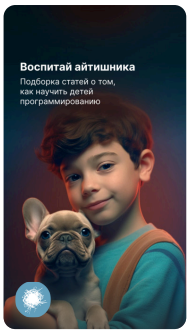


Лучшее из блогов компаний

Остановка «Ретейл будущего»

Там на неведомых дорожках...

Схватил за мозг



Как учить детей программированию

При поддержке Хабр Курсы РЕКЛАМА

При поддержке Хабр Курсы РЕКЛАМА

БЛИЖАЙШИЕ СОБЫТИЯ

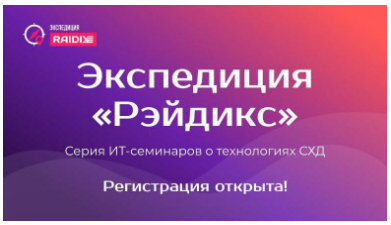
2 марта – 10 августа

PROBA Awards — международная коммуникационная премия, учрежденная в 2000 году

Санкт-Петербург

Маркетинг

Больше событий в календаре



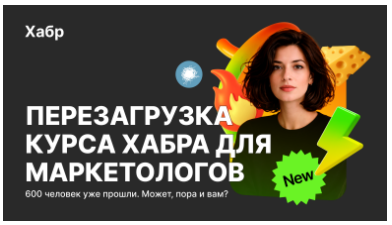
3 марта – 9 июня

Экспедиция «Рэйдикс»

Екатеринбург • Новосибирск • Краснодар • Минск • Калининград • Ростов-на-Дону

Разработка Другое

Больше событий в календаре



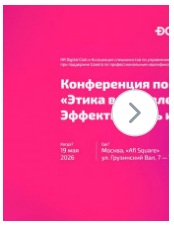
9 марта – 1 июня

Всё, что нужно, чтобы стартовать продвижение бренда на Хабре

Онлайн

Маркетинг

Больше событий в календаре



19 мая

Первая в Р конференц Этике

Москва

Менеджмент

Больше событий

При поддержке Хабр Курсы РЕКЛАМА

При поддержке Хабр Курсы РЕКЛАМА

При поддержке Хабр Курсы РЕКЛАМА

При поддержке Хабр Курсы РЕКЛАМА

При поддержке Хабр Курсы РЕКЛАМА

© 2006–2026, Habr